

CV2 Model					
กลุ่มงาน	“โมเดล/อัลกอริทึม” ใน cv2	เรียกใช้ผ่าน	ใช้ทำอะไร	จุดเด่น	ข้อจำกัด/หมายเหตุ
ตรวจจับวัตถุ/ใบหน้า (Classic)	Haar Cascade	cv2.CascadeClassifier	ตรวจจับหน้า/ตา/รอยยิ้ม/วัตถุแบบง่าย	เร็ว เบา ใช้ CPU ได้ดี	ความแม่นยำตกเมื่อแสง/มุม/ระยะเปลี่ยนมาก
	LBP Cascade	cv2.CascadeClassifier	ตรวจจับแบบคล้าย Haar แต่ใช้ LBP feature	เร็วกว่า Haar บางเคส	แม่นยำ DL ไม่ได้
ตรวจจับคนเดินเท้า	HOG + SVM (People Detector)	cv2.HOGDescriptor_getDefaultPeopleDetector()	ตรวจจับคนเต็มตัวในภาพ	ไม่ต้องเทรนเอง ใช้ได้ทันที	จำกัดกับ “คนเดินเท้า” และสภาพแวดล้อมพอสมควร
ตรวจจับวัตถุ/ใบหน้า (Deep Learning)	DNN (โหลดโมเดลภายนอก เช่น SSD/YOLO/ONNX)	cv2.dnn.readNet...	ตรวจจับวัตถุ, face detector แบบ DL	แม่นยำกว่า classic มาก	ต้องมีไฟล์โมเดล/คอนฟิกเอง (เช่น .onnx, .pb, .caffemodel)
ติดตามวัตถุ (Tracking)	CSRT Tracker	cv2.TrackerCSRT_create()	ติดตามกรอบวัตถุ (เริ่มจากกรอบแรก)	แม่นยำ/ทนการเปลี่ยนแปลงดีกว่า KCF	ช้ากว่า เหมาะวัตถุไม่เยอะ
	KCF Tracker	cv2.TrackerKCF_create()	ติดตามกรอบวัตถุ	เร็วกว่า CSRT	หลุดง่ายเมื่อบัง/เปลี่ยนสเกลมาก
	MOSSE Tracker	cv2.TrackerMOSSE_create()	ติดตามกรอบวัตถุ	เร็วมาก เหมาะเรียลไทม์	แม่นยำ/หลุดง่าย
	MIL Tracker	cv2.TrackerMIL_create()	ติดตามกรอบวัตถุ	ใช้ง่าย	มักแพ้ CSRT/KCF ในความแม่นยำ
แยกวัตถุเคลื่อนที่	Background Subtraction MOG2	cv2.createBackgroundSubtractorMOG2()	แยก foreground จากพื้นหลัง (CCTV, นับคน/รถ)	ปรับตัวกับฉากได้	แพ้ง่าย/สิ้น/แสงเปลี่ยนเร็ว ต้องจูน
	Background Subtraction KNN	cv2.createBackgroundSubtractorKNN()	แยก foreground คล้าย MOG2	บางฉากนิ่ง ๆ ทำได้ดี	ต้องจูนเหมือนกัน
การเคลื่อนไหว (Motion)	Optical Flow Farneback	cv2.calcOpticalFlowFarneback	หาเวกเตอร์การเคลื่อนไหวทั้งภาพ	ได้ motion field ทั้งภาพ	หนักกว่า/ไวต่อ noise
	Optical Flow Lucas–Kanade (KLT)	cv2.calcOpticalFlowPyrLK	ตาม “จุด” ที่เลือก/ตรวจพบ	เร็ว เหมาะ tracking จุด	ต้องมีจุดดี ๆ และอาจหลุด
จุดเด่น/จับคู่ภาพ	ORB	cv2.ORB_create()	หา keypoints+descriptors จับคู่ภาพ	ฟรี เร็ว เหมาะงานทั่วไป	แม่นยำ SIFT บางงานไม่ได้
	AKAZE	cv2.AKAZE_create()	feature สำหรับจับคู่ภาพ	ทน scale/blur ดีบางเคส	ช้ากว่า ORB บางงาน
	SIFT (แล้วแต่ build)	cv2.SIFT_create()	feature คุณภาพสูงสำหรับ matching	แม่นยำ/เสถียรมากกว่า ORB หลายงาน	หนักกว่า (และบางเครื่องอาจไม่มี)
จับคู่/เรขาคณิต	Homography / RANSAC	cv2.findHomography	หาแปลงมุมมอง (panorama, AR)	ใช้ร่วมกับ matching ได้ดี	ต้องมีจุด match ที่ดีพอ
คาลิเบรตกล้อง	Camera Calibration	cv2.calibrateCamera	หา intrinsics/distortion	แก้ภาพบิด, วัดระยะ	ต้องมีภาพ chessboard/charuco ที่ดี
แก้ distortion	Undistort / Remap	cv2.undistort, cv2.initUndistortRectifyMap	แก้ภาพบิดเลนส์	ทำให้ geometry ตรงขึ้น	ต้องมีค่าคาลิเบรต
Pose Estimation	PnP	cv2.solvePnP	หา pose (R,t) จาก 2D-3D	ใช้กับ marker/AR/วัตถุตำแหน่ง	ต้องมีจุด 3D reference/การแมตช์ที่ดี
Marker (Fiducial)	ArUco / ChArUco (มักเป็น contrib)	cv2.aruco	ตรวจ marker, หา pose	เสถียร ใช้งานง่าย	ต้องติดตั้ง opencv-contrib-python
Face Recognition (Classic)	LBPH (contrib)	cv2.face.LBPHFaceRecognizer_create()	จดจำใบหน้าแบบคลาสสิก	ใช้ข้อมูลน้อยก็พอได้	สู้ deep face embedding ไม่ได้
Image Segmentation (DL)	DNN Segmentation (โมเดลภายนอก)	cv2.dnn	แบ่งส่วนภาพ (เช่น คน/ฉาก)	แม่นยำถ้าใช้โมเดลดี	ต้องมีโมเดลเอง + อาจหนัก

CV2 Command

ขั้นตอน	คำสั่ง (OpenCV)	ใช้ทำอะไร	ได้ค่า/ผลลัพธ์	หมายเหตุ (จูนบ่อย)
อ่านภาพ	cv2.imread()	โหลดภาพ	img	ถ้า None = path ผิด
แปลงสี	cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)	ทำเป็นภาพเทาให้ทำ edge ง่าย	gray	ถ้าใช้แยกสี ให้คง BGR/ไป HSV แทน
ลด noise	cv2.GaussianBlur(gray, (k,k), 0)	ลดจตุรบกวนก่อนหาเส้นขอบ	blur	k มัก 3/5/7
หาเส้นขอบ	cv2.Canny(blur, t1, t2)	สร้างภาพเส้นขอบ	edges	t1,t2 มีผลมาก (เช่น 50,150)
หา contour	cv2.findContours(edges, mode, method)	หาเส้นขอบเป็นรูปร่าง	contours	mode: RETR_EXTERNAL เอาเฉพาะขอบนอก
กรองด้วยพื้นที่	cv2.contourArea(c)	วัดขนาดรูปร่าง	area	ตัดของเล็ก ๆ เช่น < 300
ความยาวรอบรูป	cv2.arcLength(c, True)	ใช้คำนวณค่า approx	peri	True = contour ปิด
ทำให้เป็นหลายเหลี่ยม	cv2.approxPolyDP(c, eps, True)	ลดจุดให้เหลือ "มุม" ของรูปทรง	approx	eps = 0.01–0.03 * peri จูนสำคัญสุด
นับจำนวนมุม	len(approx)	ตัดสินรูปทรง (3=สามเหลี่ยม, 4=สี่เหลี่ยม)	v	ถ้าแสง/ขอบไม่ชัด อาจได้ 5–8 ต้องจูน eps/threshold
ครอบกรอบสี่เหลี่ยม	cv2.boundingRect(approx)	หา x,y,w,h	(x,y,w,h)	ใช้คำนวณอัตราส่วน (square/rectangle)
เช็คสี่เหลี่ยมจัตุรัส	ar = w/float(h)	แยก Square vs Rectangle	ar	เช่น 0.95–1.05 = square (จูนตามงาน)
วาดเส้น contour	cv2.drawContours(img, [approx], -1, color, thick)	วาดขอบรูปทรงที่ตรวจพบ	ภาพที่มีเส้น	ใช้ดูว่ามุมที่หาได้ถูกไหม
วาดกรอบ	cv2.rectangle(img, (x,y), (x+w,y+h), color, thick)	วาดกรอบล้อมวัตถุ	ภาพมีกรอบ	ใช้แสดงผล
ใส่ชื่อ	cv2.putText(img, label, (x,y), font, scale, color, thick)	แปะข้อความ Triangle/Rectangle	ภาพมีข้อความ	ปรับ scale ให้พอดี
บันทึก	cv2.imwrite("out.png", img)	เซฟผลลัพธ์	ไฟล์ภาพ	—
(ตัวเลือก) แยกสี	cv2.cvtColor(img, cv2.COLOR_BGR2HSV) + cv2.inRange()	แยกสีวัตถุให้ชัดก่อนหา contour	mask	เหมาะมากถ้าวัตถุเป็นสีเด่น ๆ
(ตัวเลือก) เก็บงานขอบ	cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)	ลบจตุรบกวน/อุดรู	mask2	ช่วยให้ contour เนียนขึ้น